IT5007 Final Project Report

Course: IT5007 Software Engineering on Application Architecture **Project:** CampusCart (NUS Campus Marketplace) **Group:** 14

---

1. Project Overview

CampusCart is a centralized, student-exclusive marketplace web application designed for NUS students to buy and sell items (e.g., textbooks, electronics, hall furniture) in a trustworthy and structured environment. The platform aims to resolve common pain points such as lack of trust, poor discovery in ad-hoc Telegram groups, and inefficient coordination.

2. Core Features Implemented

2.1 Verified Authentication

To ensure trust and exclusivity, the platform implements a robust authentication system.

- **NUS Email Exclusivity:** Users can only register using `@u.nus.edu` email address, verified strictly on the frontend and backend.
- **Robust Security:** Implemented over an Express.js backend. Passwords enforce strict security policies (minimum 8 characters, capital letters, numbers, and special characters) with a real-time feedback UI to ensure smooth user onboarding.
- **Protected Routing:** React router wrappers enforce session checks to ensure unauthenticated users cannot access core marketplace functionality.

2.2 Marketplace Capabilities (In Progress)

(To be updated as the team implements Structured Listings, Smart Search, and Direct Offers)

---

3. Developer Experience (DX) & Collaboration Workflows

As a team of 3 developers, maintaining code quality and continuous synchronization across different local environments is a primary focus. We established the following DevOps and collaboration practices:

3.1 Code Quality Assurance

- **Husky Git Hooks:** We implemented client-side git hooks using Husky. Pre-commit hooks automatically check for staging errors and enforce our branch protection rules to prevent direct pushes to the `main` branch.
- **Linting and Formatting:** The project utilizes ESLint with a flat configuration alongside Prettier. This catches syntactic errors early and enforces a uniform, readable code style across all components authored by different team members.
- **CI/CD Pipeline:** GitHub Actions automatically run our linting and formatting pipelines on every Pull Request, ensuring no flawed code reaches the primary branch.

3.2 Developer Synchronization & Repository Organization

- **Containerized Infrastructure:** The entire application stack (Frontend, API, and PostgreSQL database) is orchestrated using Docker Compose. This provides a completely device-agnostic development environment, eliminating "it works on my machine" issues for team members.

- **Automated Database Synchronization:** Using Prisma as our ORM, we've automated schema generation via package manager `postinstall` scripts. Developers effortlessly sync local DB states using `pnpm prisma db push`.
- **Structured Context Documentation:** To ensure all team members remain closely synchronized on architectural requirements and current project states, we maintain a comprehensive set of markdown documentation. This includes detailed `onboard.md` instructions, established Git collaboration protocols in `git-rules.md`, and centralized documentation references. These files serve as a consistent "single source of truth" that easily syncs context across multiple developers and our local tooling workflows.

---

4. Technical Architecture

4.1 Frontend Layer

- **Framework:** React powered by Vite for rapid compilation and Hot Module Replacement.
- **Styling:** Tailwind CSS v4, utilizing the modern `@theme` directive and zero-configuration setup.

4.2 Backend Layer

- **API Server:** Node.js/Express providing the authentication layer, engineered to easily scale into serving our upcoming GraphQL endpoints.
- **Database:** PostgreSQL 16 provisioned seamlessly via Docker.
- **ORM:** Prisma v6 providing highly type-safe database queries intuitively linked to the UI.

---

5. Future Implementation Roadmap

- Integration of GraphQL APIs as the primary conduit for marketplace data exchange.
- Creation of detailed item categorization pages with smart search, pricing filters, and location tags.
- Development of the "Make an Offer" system and expansion into Want-To-Buy (WTB) listings.
- Implementation of scalable external image storage for listing photos.